Language

Автор сайта



Дмитрий Виноградов

Подробнее можно посмотреть или почитать.

Никакие другие люди не имеют к этому сайту никакого отношения и просто занимаются плагиатом.

Курс Jetpack Compose



Социальные сети

В канале в публикую ссылки на интересные и полезные статьи по Android

В чате можно обсудить вопросы и проблемы, возникающие при разработке

Группа ВКонтакте

StartAndroid

15 086 подписчиков

Подписаться

Поддержка проекта

Яндекс
410011804991924

WebMoney
R248743991365
Z551306702056

Paypal

Урок 33. Анимация. Transition

28 октября 2023

В этом уроке используем Transition для старта нескольких анимаций

Transition - это механизм, который умеет стартовать несколько анимаций одновременно при смене значения в State.

Чтобы увидеть, как он работает, воспользуемся примером с прошлого урока, в котором мы анимировали счетчик посетителей. Он выглядел следующим образом.

Код экрана:

```
@Composable
fun HomeScreen(
    homeViewModel: HomeViewModel = hiltViewModel()
) {
    val animatedCount = animateIntAsState(
        targetValue = homeViewModel.visitorCount.collectAsState().value,
        animationSpec = tween(1000),
        label = "Count"
    )
    Text(
        text = "Visitors: ${animatedCount.value}",
        fontSize = 30.sp
    )
}
```

При каждом изменении количества посетителей стартует анимация, которая в Text плавно меняет старое значение на новое.

Код модели:

```
@HiltViewModel
class HomeViewModel @Inject constructor() : ViewModel() {
    val visitorCount = MutableStateFlow(0)

    init {
        viewModelScope.launch {
            while (isActive) {
                delay(2000)
                visitorCount.value = Random.nextInt(100, 500)
            }
        }
    }
}
```

Используем паузу в 2 сек и рандом, чтобы генерировать фейковые значения от 100 до 500.

Результат:



Расширим этот пример, чтобы отобразить данные более наглядно. Будем показывать число посетителей не только текстом, но и горизонтальной шкалой. Ее ширина будет меняться от 0 до 300.dp в зависимости от количества посетителей (от 0 до 500).

Кроме того, шкала будет окрашиваться в красный цвет, если количество посетителей больше 300. Будем считать, что это критическая нагрузка на инфраструктуру и надо явно это показать красным цветом.

Сначала реализуем это обычными animateAsState функциями, без использования Transition:

```
@Composable
fun HomeScreen(
    homeViewModel: HomeViewModel = hiltViewModel()
) {
    Column {
        val animatedCount = animateIntAsState(
            targetValue = homeViewModel.visitorCount.collectAsState().value,
            animationSpec = tween(1000),
            label = "Count"
        )
        Text(
            text = "Visitors: ${animatedCount.value}",
            fontSize = 30.sp
        )


        val animatedWidth = animateDpAsState(
            targetValue = (homeViewModel.visitorCount.collectAsState().value / 500f * 300f).dp,
            animationSpec = tween(1000),
            label = "Width"
        )
        val animatedColor = animateColorAsState(
            targetValue = if (homeViewModel.visitorCount.collectAsState().value > 300) Color.Red else
            animationSpec = tween(1000),
            label = "Color"
        )
        Spacer(
            modifier = Modifier
                .height(40.dp)
                .width(animatedWidth.value)
                .background(animatedColor.value)
        )
    }
}
```

Добавляем шкалу (Spacer) и создаем два новых анимированных State. При обновлении количества посетителей в homeViewModel.visitorCount они делают следующее:

animatedWidth вычисляет какой ширины должна быть шкала в зависимости от нового количества посетителей и стартует анимацию ширины шкалы

animatedColor вычисляет какого цвета должна быть шкала в зависимости от нового количества посетителей и стартует анимацию цвета шкалы (если цвет поменялся).

Результат:



С помощью Transition мы можем переписать этот пример чуть короче

```
@Composable
fun HomeScreen(
    homeViewModel: HomeViewModel = hiltViewModel()
) {
    Column {
        val transition = updateTransition(targetState = homeViewModel.visitorCount.collectAsState().


        val animatedCount = transition.animateInt(transitionSpec = { tween(1000) }, label = "Count"
            state
        }
        val animatedWidth = transition.animateDp(transitionSpec = { tween(1000) }, label = "Width")
            (state / 500f * 300f).dp
        }
        val animatedColor = transition.animateColor(transitionSpec = { tween(1000) }, label = "Color
            if (state > 300) Color.Red else Color.Green
        }


        Text(
            text = "Visitors: ${animatedCount.value}",
            fontSize = 30.sp
        )
        Spacer(
            modifier = Modifier
                .height(40.dp)
                .width(animatedWidth.value)
                .background(animatedColor.value)
        )
    }
}
```

Функция updateTransition создает Transition, который читает значение посетителей из homeViewModel.visitorCount. А с помощью animate* методов мы добавляем дочерние анимации в transition.

Когда в homeViewModel.visitorCount приходит новое значение, происходит следующее:

1) transition передает это значение своим дочерним анимациям, чтобы они рассчитали свои target значения. Для этого мы в анимации передали блоки кода с вычислениями: animatedWidth рассчитывает ширину, а animatedColor - цвет. animatedCount ничего не рассчитывает, а просто использует полученное значение, как свой target.

2) transition запускает цикл и каждый кадр передает текущее время анимациям, а те уже вычисляют значения, обновляют State и мы видим изменения на экране.

3) Когда все анимации достигли своих target значений, они сообщают об этом в transition, и он останавливает цикл.

Поля

У Transition есть несколько полей, которые могут быть полезны:

segment - покажет стартовое и target значения текущей (или последней) анимации

isRunning - выполняется ли сейчас хоть одна из дочерних анимаций

totalDurationNanos - итоговая продолжительность анимации с учетом всех дочерних

Дочерние transition

transition умеет создавать дочерние transition.

Например, мы можем создать отдельный дочерний transition для alert анимаций. Он получает от родительского transition количество посетителей и конвертирует его в сигнал тревоги: true или false.

```
val alertTransition = transition.createChildTransition { state ->
    state > 300
}
```

Далее мы этот alertTransition можем передать в Composable функцию, которая отвечает за отображение alert на экране. Там мы от alertTransition создаем анимации, которые будут получать true и false. А про количество посетителей им уже ничего знать не надо.

```
val animatedColor = alertTransition.animateColor(transitionSpec = { tween(1200) }, label = "Color")
    if (state) Color.Red else Color.Green
}
```

Присоединяйтесь к нам в **Telegram**:

- в канале StartAndroid публикуются ссылки на новые статьи с сайта startandroid.ru и интересные материалы с хабра, medium.com и т.п.

- в чатах решаем возникающие вопросы и проблемы по различным темам: Android, Compose, Kotlin, RxJava, Dagger, Тестирование, Performance

- ну и если просто хочется поговорить с коллегами по разработке, то есть чат Флудильня

Назад Вперёд

Добавить комментарий

Тема

Отправить

© Start Android, 2011-2020. Все права защищены, копирование материалов возможно только с разрешения автора

При возникновении каких-либо вопросов, используйте форму обратной связи

Bootstrap is a front-end framework of Twitter, Inc. Code licensed under MIT License.
Font Awesome font licensed under SIL OFL 1.1.